

Computational Physics

Summer 2026

Lecture #17

12. 6. 2026

Dylan Nelson

Review (Euler's Method)

given a first order ODE :

$$y' = f(t, y)$$

the Taylor expansion at t_i , evaluated at t_{i+1}

$$y(t_{i+1}) = y(t_i) + \underbrace{(t_{i+1} - t_i)}_{\text{stepsize } h} y'(t_i) + \dots \quad \text{drop } \mathcal{O}(h^2)$$

$$\Rightarrow \text{approximate solution } \tilde{y}_{i+1} = \tilde{y}_i + h y'(t_i) \\ = \tilde{y}_i + h f(t_i, \tilde{y}_i)$$

global truncation error $\sim \mathcal{O}(h) \Rightarrow$ Euler's method is "first order"

Higher Order - Taylor Expansion

for $y(t) \in C^{n+1}[a, b]$ with $(n+1)$ continuous derivatives,

- expand $y(t)$ with its n^{th} Taylor polynomial about t_i
- evaluate at t_{i+1}

$$y(t_{i+1}) = y(t_i) + h y'(t_i) + \frac{h^2}{2} y''(t_i) + \dots + \frac{h^n}{n!} y^{(n)}(t_i) + \frac{h^{n+1}}{(n+1)!} y^{(n+1)}(\xi_i)$$

for some number $\xi_i \in [t_i, t_{i+1}]$

- differentiating :

$$y'(t) = f(t, y)$$

$$y''(t) = f'(t, y) = \frac{d}{dt} f(t, y) + \frac{d}{dy} f(t, y) \cdot \frac{dy}{dt} = f_t + f_y f$$

$y^{(k)}(t) = f^{(k-1)}(t, y)$, substitute into the above :

$$y_{i+1} = y_i + h f(t_i, y_i) + \frac{h^2}{2} f'(t_i, y_i) + \dots + \frac{h^n}{n!} f^{(n-1)}(t_i, y_i) + \frac{h^{n+1}}{(n+1)!} f^{(n)}(t_i, y_i)$$

$\hookrightarrow$ drop remainder term $\equiv$ Taylor method of order n

Taylor method of order n :

(2)

$$\tilde{y}_{i+1} = \tilde{y}_i + h T^n(t_i, \tilde{y}_i)$$

$$\text{where } T^n(t_i, \tilde{y}_i) = f(t_i, \tilde{y}_i) + \frac{h}{2} f'(t_i, \tilde{y}_i) + \dots + \frac{h^{n-1}}{n!} f^{(n-1)}(t_i, \tilde{y}_i)$$

note: Euler's method $\equiv$ Taylor method of order one

Thm: local truncation error $\sim \mathcal{O}(h^{n+1})$

pro: can achieve high-order i.e. rapid convergence for $h \rightarrow 0$

con: must compute derivatives, usually difficult/expensive

note: all Taylor methods are one-step

$\Rightarrow$ calculating $\tilde{y}(t_{i+1})$ uses only information at t_i

note, since $f(t, y)$ is a function of both t and $y(t)$:

$$f' = \frac{d}{dt} f(t, y) = \frac{d}{dt} f(t, y) + \frac{d}{dy} f(t, y) \cdot \frac{dy}{dt} = \frac{df}{dt} + \frac{df}{dy} \cdot f$$

introduce notation:

$$f_t = \frac{df}{dt}, \quad f_y = \frac{df}{dy}, \quad f_{ty} = \frac{d^2 f}{dt dy} = \frac{d^2 f}{dy dt} = f_{yt}, \dots$$

then in general:

$$f^{(k)}(t, y) = \frac{d}{dt} f^{(k-1)}(t, y) + \frac{d}{dy} f^{(k-1)}(t, y) \cdot \frac{dy}{dt} = f_t^{(k-1)} + f_y^{(k-1)} f$$

such that, to write out explicitly the first few terms:

$$\begin{aligned} y_{i+1} = y_i + hf + \frac{h^2}{2} (f_t + f f_y) + \frac{h^3}{3!} (f_{tt} + 2f_{ty} + f^2 f_{yy} + f f_y^2 + f_t f_y) \\ + \mathcal{O}(h^4) \end{aligned}$$

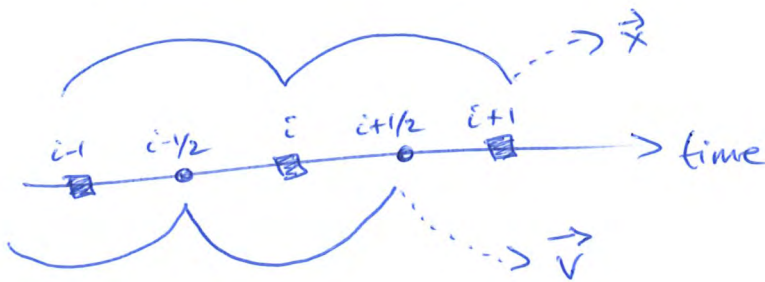
Multi-Step Methods : Leap Frog

goal: solve ODE of the form $x''(t) = \frac{d^2x}{dt^2} = A(x)$

$$\Rightarrow \begin{cases} v'(t) = A(x) \\ x'(t) = v(t) \end{cases}$$

idea: take information from two previous timesteps

- (1) update velocity : $v_{i+1/2} = v_{i-1/2} + a_i h + \mathcal{O}(h^3)$
 (on half timesteps) $\uparrow$ full-timestep acceleration, $a_i = A(x_i)$
- (2) update position : $x_{i+1} = x_i + v_{i+1/2} h + \mathcal{O}(h^3)$
 (on full timesteps) $\uparrow$ half timestep i.e. midpoint velocity



KDK version

$$\begin{aligned} v_{i+1/2} &= v_i + a_i \frac{h}{2} && \text{"kick"} \Delta t/2 \\ x_{i+1} &= x_i + v_{i+1/2} h && \text{"drift"} \Delta t \\ v_{i+1} &= v_{i+1/2} + a_{i+1} \frac{h}{2} && \text{"kick"} \Delta t/2 \\ &&& (\text{and repeat}) \end{aligned}$$

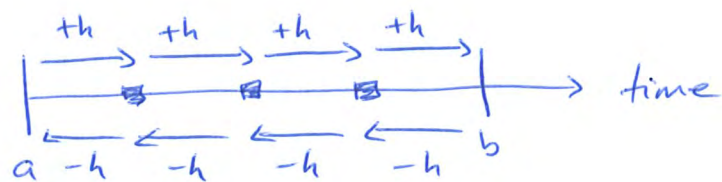
Q: Why is leapfrog second order?

$$\epsilon_{\text{local}} \sim \mathcal{O}(h^3) \Rightarrow \epsilon_{\text{global}} \sim \mathcal{O}(h^2)$$

$\Rightarrow$ asynchronous integration of (x, v) makes this a symplectic integrator

Symplectic Integrators

Def: time-reversible $\Rightarrow$ conserve total energy 'exactly'



Ex: integrate from state $(x_i, v_{i-1/2})$ to $(x_{i+1}, v_{i+1/2})$, i.e. forwards
then from $(x_{i+1}, v_{i+1/2})$ to $(x_i, v_{i-1/2})$ i.e. backwards

$$x_{i+1} = x_i + v_{i+1/2} h$$

$$x_{\text{final}} = x_{i+1} + v_{i+1/2} (-h) = [x_i + v_{i+1/2} h] - v_{i+1/2} h = x_i \quad \checkmark$$

compare to e.g. the forward Euler method instead:

$$x_{i+1} = x_i + h v_i$$

$$x_{\text{final}} = x_{i+1} + (-h) v_{i+1} = [x_i + h v_i] - h v_{i+1} = x_i + h \underbrace{(v_i - v_{i+1})}$$

in general, $v_i \neq v_{i+1}$

$$\Rightarrow x_{\text{final}} \neq x_i$$

Thm: symplectic integrators conserve:

(1) integrals of motion & integral invariants
e.g. total energy

(2) phase-space density, along trajectories of solutions (Liouville's theorem)

Def: Hamiltonian system

$\begin{pmatrix} \vec{q} \\ \vec{p} \end{pmatrix} \equiv \begin{pmatrix} \text{generalized position} \\ \text{momentum} \end{pmatrix}$ is a state vector of "canonical coordinates"

$$\dot{\vec{p}} = \frac{d\vec{p}}{dt} = -\frac{\partial H}{\partial \vec{q}} \quad \dot{\vec{q}} = \frac{d\vec{q}}{dt} = \frac{\partial H}{\partial \vec{p}}$$

Thm: a symplectic integrator is a canonical transformation of a Hamiltonian system

$(\vec{q}, \vec{p}) \rightarrow (\vec{q}', \vec{p}')$ preserves Hamilton's equations

$\Rightarrow$ provides an 'exact' solution

note: for many dynamical systems, $H(\vec{p}, \vec{q}) = H_{\text{kin}}(\vec{p}) + H_{\text{pot}}(\vec{q})$ separable

$\Rightarrow$ operator splitting,

apply Hamilton's equations to each separately

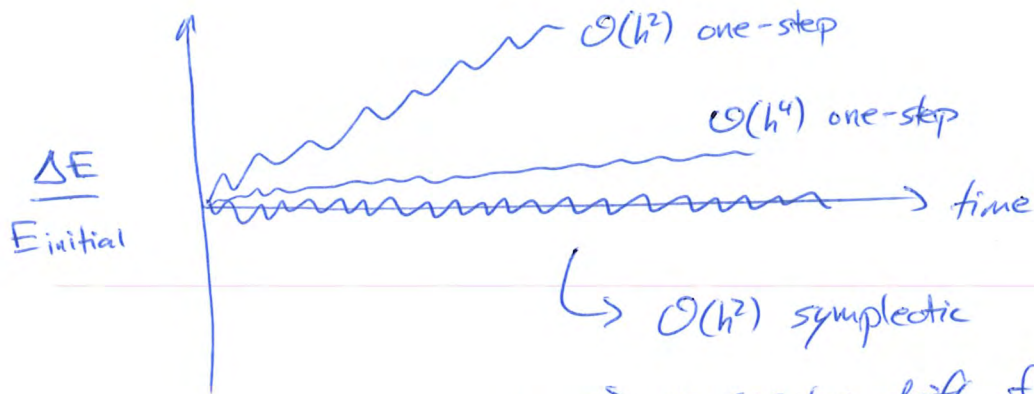
$$\underbrace{\frac{1}{2} m v^2}_{\vec{p}} = \frac{\vec{p}^2}{2m} \quad \underbrace{U_{\text{grav}}(\vec{q})}_{\vec{q}} \sim -\frac{Gm}{r}$$

Thm: leapfrog is symplectic because it is a composition of symplectic steps on H_{kin} and H_{pot} separately

note: in practice, leapfrog solves an approximate

$$\tilde{H}(\vec{p}, \vec{q}) = H_{\text{kin}}(\vec{p}) + H_{\text{pot}}(\vec{q}) + H_{\text{err}}$$

where $H_{\text{err}} \sim \mathcal{O}(h^2)$ is a numerical error Hamiltonian



$\hookrightarrow \mathcal{O}(h^2)$ symplectic

$\Rightarrow$ no secular drift of E_{tot}

$\Rightarrow$ good for long-term integration